LeRobot 平台感知—推理—执行流程的系统级性能剖析研究

毕业论文大纲（草稿）

2026 年 4 月 19 日

目录

1 绪论 **3**

1.1 研究背景与意义 . . . 3

1.2 研究目标 . . . 3

1.3 主要贡献 . . . 3

1.4 论文组织结构 . . . 3

2 相关工作 **3**

2.1 机器人操作系统与框架 . . . 3

2.2 嵌入式边缘 AI 推理 . . . 4

2.3 ACT 与 VLA 模型 . . . 4

2.4 系统级 Profiling 技术 . . . 4

3 系统架构与负载特征分析 **4**

3.1 LeRobot 软件栈架构 . . . 4

3.2 三类典型工作负载特征 . . . 5

3.3 感知—推理—执行三阶段划分 . . . 5

4 跨层次性能分析模型与剖析方法 **5**

4.1 四层性能分析模型 . . . 5

4.2 工具链选型 . . . 5

4.3 实验配置 . . . 6

5 系统级性能剖析结果 **6**

5.1 感知阶段 . . . 6

5.1.1 摄像头路径 . . . 6

5.1.2 舵机路径 . . . 6

5.2 推理阶段 7
5.2.1 整体耗时分布 7
5.2.2 算子级耗时分解 7
5.2.3 PyTorch 线程行为 7
5.3 执行阶段 8
5.3.1 sync_write 路径 8
5.3.2 异步推理流水线分析 8
5.4 全链路综合分析 8
5.4.1 时间预算表（单帧，33.3 ms 目标） 8
5.4.2 系统调用分布（40 s episode） 9

6 性能瓶颈分析与优化建议 9
6.1 瓶颈归因 9
6.2 优化方向分析 9
6.3 软件架构改进方向 9

7 总结与展望 10
7.1 主要工作总结 10
7.2 研究局限性 10
7.3 未来工作 10

附录：需要补充的新实验 11

1 绪论

1.1 研究背景与意义

- 协作机器人与 VLA 模型快速发展，软件栈日趋复杂；
- 主流机器人系统基于通用宏内核构建，与”感知—推理—执行”紧耦合运行特征存在明显差异；
- 引入 ACT 等 AI 推理框架后，系统性能瓶颈与资源消耗机理缺乏系统性分析。

1.2 研究目标

1. 建立跨层次（AI 框架层 / 运行时层 / OS 内核层 / 硬件 I/O 层）性能分析模型；
2. 端到端量化感知、推理、执行各阶段性能开销；
3. 定位主要瓶颈，为系统优化与软件架构改进提供实验依据。

1.3 主要贡献

- 提出覆盖 AI 框架到内核的四层性能分析模型；
- 通过 ftrace/torch.profiler/perf 多工具联合剖析，量化各阶段时间开销；
- 揭示 strace 对系统调用测量带来的 9× 时间放大问题，验证 ftrace 的必要性；
- 基于量化结果，系统梳理优化路径及其成本收益。

1.4 论文组织结构

各章内容简述如下（第 2–7 章）。

2 相关工作

2.1 机器人操作系统与框架

- ROS2、LeRobot、LCM 对比——通信机制与实时性；
- LeRobot v0.3.4 架构概述：record/replay/train/eval 流程。

2.2 嵌入式边缘 AI 推理

- TFLite、ONNX Runtime、PyTorch Mobile 在 ARM 上的部署现状；
- 树莓派 5（Cortex-A76）的计算能力与限制。

2.3 ACT 与 VLA 模型

- ACT 模型原理：CVAE 编解码器、动作分块（chunk）、time ensemble；
- chunk_size 与推理频率的权衡。

[来源：ACT 模型原理.md，ACT 推理.md]

2.4 系统级 Profiling 技术

- strace / ftrace / perf / torch.profiler 工具对比；
- 已有机器人系统性能分析工作综述。

3 系统架构与负载特征分析

3.1 LeRobot 软件栈架构

五层结构：

层级	主要组件
应用层	record.py / RecordRunner / lerobot-* CLI
AI 框架层	ACT policy / ResNet18 / Transformer
运行时层	PyTorch / ATen / OpenMP 线程池 / OpenCV
内核层	futex / pselect6 / ioctl / V4L2 / TTY
硬件层	USB 摄像头 / Feetech SCServo / ttyACM0

关键代码入口：`lerobot/src/lerobot/record.py`，核心循环：`get_observation()` → `select_action()` → `send_action()`。

[来源：LeRobot 软件栈全链路分析.md]

3.2 三类典型工作负载特征

[新实验：各跑一次 episode，用 Python logging 采集帧率、推理次数、观测维度]

三类任务：**抓取（Grasp）、推倒（Push）、分类（Classify）**。

指标	抓取	推倒	分类
Episode 时长（s）	**[TODO：测量]**	**[TODO：测量]**	**[TODO：测量]**
推理调用次数	**[TODO：测量]**	**[TODO：测量]**	**[TODO：测量]**
摄像头路数	2	2	2
关节自由度	6	6	6

3.3 感知—推理—执行三阶段划分

- 阶段边界（代码级）：`get_observation()` / `select_action()` / `send_action()`；
- 时间预算目标：30 fps ⇒ 33.3 ms/帧；
- 当前实测：单帧循环 2.2–4.2 s（推理主导）。

4 跨层次性能分析模型与剖析方法

4.1 四层性能分析模型

层级	可观测指标	工具
AI 推理框架层	算子耗时、张量形状、FLOP	torch.profiler
Python 运行时层	GIL 争用、帧循环时间	Python logging
OS 内核层	系统调用次数/耗时、线程调度	ftrace, strace
硬件 I/O 层	传输字节数、等待时延	perf, ioctl trace

4.2 工具链选型

strace vs ftrace 的关键差异：

- strace 通过 ptrace 拦截每次系统调用，带来约 **9× 系统时间放大**（实测：20.12 s → 2.20 s）；
- ftrace 工作在内核 tracepoint，开销极低，适合生产级测量；
- **本文以 ftrace 数据为准，strace 仅用于初期探索。**

[来源：内核数据搜集分析.md，方法论.md]

4.3 实验配置

- 硬件：树莓派 5（ARM Cortex-A76，4 核，8 GB LPDDR4X）；
- 软件：LeRobot v0.3.4 / PyTorch 2.x / Python 3.10 / Ubuntu 24.04；
- 关键参数：`chunk_size=100`，fps=30，串口波特率 =1 Mbps；
- 每类任务采集 ≥ 3 次 episode 取均值，排除冷启动影响（模型加载 114 万次 read 调用仅在第一次出现）。

5 系统级性能剖析结果

5.1 感知阶段

5.1.1 摄像头路径

调用链：`cv2.VideoCapture.read()` → `grabFrame`（VIDIOC_DQBUF） → `retrieveFrame`（libjpeg 解码）。

- pselect6 行为：摄像头后台线程（TID 4222/4224）阻塞 **23–32 ms**，符合 30 fps 帧间隔物理极限；
- MJPEG 解码耗时：**10–30 ms/帧**（libjpeg CPU 密集，是摄像头路径主要开销）；
- 整体摄像头贡献观测耗时约 **1.9 ms**（主线程视角，后台异步）。

[来源：pselect6 最终结果.md，OpenCV 视频流数据流分析.md]

5.1.2 舵机路径

调用链：`motors_bus.sync_read()` → `GroupSyncRead` → `PortHandler` → `/dev/ttyACM0`。

- Sync Read 包格式：`[0xFF 0xFF 0xFE LEN 0x82 ADDR LEN` $\mathtt{ID}_1$`..`$\mathtt{ID}_6$ `CHK]`；
- Present_Position 寄存器地址 56，sign-magnitude 编码（bit15 为方向位）；
- 耗时：**1.2 ms/帧**，开销可忽略。

[来源：observation 读取 ACM0 数据完整流程.md，Lerobot 内舵机读写完整链路.md]

5.2 推理阶段

5.2.1 整体耗时分布

- chunk 切换帧：推理耗时 **2000–4000 ms**（偶发峰值 2148 ms）；
- chunk 内帧：从 `_action_queue` 取缓存，<1 ms；
- 推理占总循环时间 >99%，是系统主瓶颈。

[来源：recordloop 每部分时间全部分析.md]

5.2.2 算子级耗时分解

[新实验：torch.profiler op-level profiling，分解 ResNet18 / Transformer Encoder / Decoder / 前后处理占比]

模块	耗时（ms）	占推理比例（%）
ResNet18 特征提取（×2 摄像头）	**[TODO：]**	**[TODO：]**
CVAE Encoder（4 层 Transformer）	**[TODO：]**	**[TODO：]**
Action Decoder（1 层 Transformer）	**[TODO：]**	**[TODO：]**
前处理（归一化、Tensor 转换）	**[TODO：]**	**[TODO：]**
后处理（queue 写入、限幅）	**[TODO：]**	**[TODO：]**
合计	**[TODO：]**	100%

5.2.3 PyTorch 线程行为

- futex 调用：**64,569 次 / 40 s episode**，总等待时间 3,252 s（27 线程并行累计）；
- op=0（FUTEX_WAIT）占多数，op=9（FUTEX_WAIT_BITSET）用于 ATen 线程池任务队列；
- ARM Cortex-A76 使用 OpenMP 并行，NEON SIMD 加速 GEMM，但计算量仍是瓶颈。

[来源：futex 最终结果.md，ACT 推理.md]

5.3 执行阶段

5.3.1 sync_write 路径

- 调用链：`motors_bus.sync_write()` → `GroupSyncWrite.txPacket()` → `write()` → TTY → USB bulk transfer；
- 无回包，半双工 TTY 总线，耗时 **1–3 ms**；
- 异步写不可行：总线半双工冲突 + 安全限幅 read-modify-write 链路断裂。

[来源：USB 串口写路径.md，motors_bus 异步读写设计分析.md]

5.3.2 异步推理流水线分析

不 stall 条件推导：

$$N \geq 2 \times T_{\text{infer}} \times F$$

代入当前参数（$T_{\text{infer}} = 2\,\text{s}$，$F = 30\,\text{fps}$，$N = 100$）：

$$\text{需要 } N \geq 120, \quad \text{当前 } N = 100 < 120 \quad \Rightarrow \text{stall 不可避免}$$

- stall 周期：20 帧卡顿 / 70 帧正常，循环出现；
- time ensemble 失效：chunk B 到达时前 50 帧已过期被丢弃；
- 增大 chunk_size 会导致动作质量下降（实验已确认），非可行方案。

[来源：async_inference_ 推理执行流水线分析.md]

5.4 全链路综合分析

5.4.1 时间预算表（单帧，33.3 ms 目标）

阶段	实测耗时	占目标比例
感知（舵机 + 图像，主线程视角）	5.2 ms	15.6%
推理（chunk 内均摊）	[TODO：] ms	[TODO：]%
执行（sync_write）	4.8 ms	14.4%
其余调度开销	[TODO：] ms	[TODO：]%
实际单帧循环（推理帧）	2200–4200 ms	**66×–126× 超时**

5.4.2 系统调用分布（40 s episode）

系统调用	次数	均值耗时	主要来源
futex	64,569	<1 μs	PyTorch 线程池 + Trae 服务
pselect6	36,191	421 ns	摄像头后台线程 V4L2 等待
ioctl	[TODO：]	[TODO：]	串口配置 + V4L2 帧队列管理
write	35,651	38 μs	USB 串口写

结论：系统调用本身开销极低（pselect6 均值 421 ns），瓶颈在用户态 CPU 推理计算，非 IO。

[新实验：CPU 频率锁定（performance governor）对推理峰值 2148 ms 的影响]

[来源：recordloop 每部分时间全部分析.md，内核数据搜集分析.md]

6 性能瓶颈分析与优化建议

6.1 瓶颈归因

- **主瓶颈**：ACT 推理（ARM CPU 矩阵乘法），2000–4000 ms/次；
- **次瓶颈**：MJPEG 解码（10–30 ms/帧，可替换为 YUYV 消除）；
- **非瓶颈确认**：系统调用本身（pselect6 421 ns，futex <1 μs，均远低于推理耗时）。

6.2 优化方向分析

优化方向	预期收益	可行性	实现复杂度
INT8/FP16 量化	推理降 2–4×	高	中
ONNX Runtime 后端	推理降 1.5–2×	高	低
异步推理(gRPC 双进程)	消除 stall（需 $T_{\text{infer}} < 1.67$ s）	需配合量化	高
MJPEG → YUYV	解码降至 0	高（相机支持）	低
异步舵机读	隐藏 1.2 ms	收益低	低
异步舵机写	**不可行**	×	—

6.3 软件架构改进方向

- 推理模块解耦（独立进程 + gRPC），避免 GIL 与主循环相互干扰；

- 摄像头格式统一为 YUYV，消除 libjpeg CPU 解码路径；
- 面向机器人负载的内核优化方向：实时调度器（SCHED_FIFO）、USB 驱动优先级提升。

7 总结与展望

7.1 主要工作总结

1. 建立了覆盖 AI 框架到内核的四层性能分析模型；
2. 通过 Python 日志 + ftrace + torch.profiler 多工具联合，端到端量化了 LeRobot 感知—推理—执行各阶段开销；
3. 揭示 ACT 推理（2000–4000 ms）为主瓶颈，系统调用 overhead 可忽略；
4. 推导了异步推理流水线不 stall 的数学条件，指出当前参数配置的局限性；
5. 系统梳理了各优化路径的可行性与成本收益。

7.2 研究局限性

- 仅在 SO101 单臂场景下测试，未覆盖双臂或移动机器人；
- 未实施量化等优化方案并验证，结论停留在分析层面；
- 无 GPU 对比基线，难以量化 CPU 推理与加速器的差距。

7.3 未来工作

- INT8 量化 + ONNX Runtime 实验，验证推理加速收益；
- 扩展至多任务场景（更多抓取类别）与多机器人平台；
- 探索面向机器人负载特征的实时操作系统设计。

附录：需要补充的新实验

优先级	实验内容	填充章节	预计耗时
★★★	torch.profiler op-level profiling	第 5.2 节推理算子分解	1 天
★★★	三类任务各跑 episode 采集日志	第 3 章负载特征对比表	1 天
★★	CPU 频率锁定对比	第 5.4 节峰值成因分析	半天
★	perf stat（IPC, cache-miss）	判断 compute vs memory bound	半天